

ANALYSE DU PROJET

Ce projet a pour objectif le développement d'une implémentation complète du jeu Tetris en utilisant le langage C++ et en appliquant les principes de la Programmation Orientée Objet (POO). L'architecture sera conçue pour être modulaire et extensible, permettant d'intégrer facilement des fonctionnalités futures telles que de nouveaux modes de jeu ou des systèmes de scoring avancés. La fonction multijoueur sera mise en place, chaque joueur disposant de son propre ordinateur, avec la possibilité de jouer contre une intelligence artificielle. L'accent sera mis sur un moteur de jeu robuste, capable de gérer avec précision les mécaniques de chute, de collision et de suppression de lignes. Le moteur graphique sera basé sur la librairie raylib, assurant un rendu rapide et une gestion simplifiée des entrées.

2. Liste des Fonctions à Implémenter

Le jeu nécessitera la mise en œuvre des fonctions essentielles suivantes, réparties entre les classes principales :

- Gestion du Jeu (**Game / Engine**) :
 - **run()** : Gère la boucle principale et la logique de jeu (gravité, mise à jour).
 - **handleInput()** : Traite les événements clavier pour le mouvement, la rotation et la chute.
 - **generateBlock()** : Sélectionné et instancié la nouvelle pièce.
 - **isGameOver()** : Vérifie la condition de fin de partie.
- Gestion du Plateau (**Grid**) :
 - **initGrid()** : Initialise la matrice 10x24 du plateau de jeu.
 - **checkCollision(Block block)** : Détecte les collisions avec les limites ou les pièces posées.
 - **insertBlock(Block block)** : Fige une pièce sur la grille après sa chute.
 - **clearLines()** : Identifie et supprime les lignes complètes.
- Gestion des Pièces (**Block / Tetrimino**) :
 - **rotate()** : Modifie l'orientation de la pièce (logique SRS).
 - **move(Direction dir)** : Calcule les nouvelles coordonnées de la pièce.
 - **draw()** : Fonction de rendu pour la pièce.
- Gestion du Score (**Stats**) :
 - **updateScore(int linesCleared)** : Met à jour le score et gère le passage au niveau suivant.
 - **strScore()** : Convertit le score en chaîne de caractères pour l'affichage.
- Multi-player (**NetworkManager**) :
 - **connectToGame(string serverAddress)** : Initialise la connexion du client au serveur (ou crée le serveur, si l'instance est désignée comme hôte).
 - **sendAction(PlayerAction action, BlockData data)** : Envoie l'action locale du joueur (mouvement, rotation, etc.) au serveur ou à l'adversaire pour la synchronisation des états de jeu.

- **receiveAction()** : Reçoit l'action d'un adversaire (ou du serveur) et met à jour la visualisation locale de sa grille.
- **synchronizeGrids()** : Assure que les deux clients (ou le client et le serveur) maintiennent le même état de jeu (essentiel pour éviter la désynchronisation *desync*).
- **disconnectPlayer()** : Gère proprement la fin de la connexion d'un joueur, libérant les ressources réseau.

3. Liste des Utilisations Possibles des Concepts de Programmation Orientée Objet

L'architecture du projet sera définie autour des concepts POO pour assurer la maintenabilité et la clarté du code :

- Encapsulation : Toutes les données sensibles (la matrice **grid** dans **Grid**, les coordonnées **position** dans **Block**, le **score** dans **Stats**) seront déclarées **private**. L'accès et la modification seront exclusivement gérés par des méthodes publiques (getters et setters), protégeant ainsi l'état interne des objets.
- Héritage : Une classe de base abstraite ou concrète (**Block** ou **Tetrimino**) sera créée pour encapsuler la logique commune à toutes les pièces (mouvement de base, couleur, etc.). Les sept formes spécifiques (I, J, L, O, S, T, Z) hériteront de cette classe de base, spécialisant uniquement leurs formes et leurs matrices de rotation.
- Polymorphisme : Le polymorphisme sera utilisé via des fonctions virtuelles. Par exemple, une méthode **draw()** ou **getTiles()** déclarée virtuellement dans la classe de base **Block** permettra à l'objet **Game** d'appeler la méthode spécifique de la pièce courante sans connaître son type exact.
- Composition : La classe principale **Game** utilisera la composition en contenant des instances de **Grid** (le plateau de jeu) et **Stats** (les données du jeu), ainsi que des pointeurs intelligents (**unique_ptr**) vers les objets **Block** actifs et suivants.